gods_eye_v1_approach_1 — Monolithic Sequential Chain Architecture

---

Title

gods_eye_v1_approach_1: Linear Chain-of-Thought Hacking Agent with Tool Augmentation

Design Period: Conceptual Iteration 1
Status: Deprecated — Superseded by Approach 2
Predecessor: None (genesis design)

Problem Statement

The foundational architectural problem this iteration attempts to solve is: **how do you give an LLM reliable, stateful access to a target system over an extended multi-step engagement without the execution context degrading or the agent losing coherence between steps?**

At this stage the problem is framed naively — as a context management problem rather than a coordination problem. The assumption is that a single sufficiently-prompted LLM, equipped with tools for shell execution and web search, can autonomously reason through a penetration testing workflow if given a structured prompt template and a long enough context window.

This leads to an architecture that is monolithic by design: one agent, one chain, one conversation thread, one memory buffer. All state lives in the LLM's context. There is no external state persistence, no agent separation of concerns, no parallelism. The agent is essentially an augmented ReAct loop.

The core structural flaw is not async handling or token limits in isolation — it is that **the architecture conflates reasoning, execution, memory, and reporting into a single stateful unit**, making every component a single point of failure and making the system's behavior entirely dependent on the LLM's ability to maintain coherence over long tool-call chains.

Proposed Architecture

Overview

A single LangChain `AgentExecutor` wraps an LLM with a curated tool set. The agent is initialized with a system prompt that encodes the penetration testing methodology (recon → enumeration → exploitation → post-exploitation → reporting). A `ConversationSummaryMemory` instance is attached to maintain context across tool calls.

The agent operates in a strict ReAct loop:

1. Receive user goal as human message.
2. Reason about what to do next (Thought).
3. Select a tool and invoke it (Action).
4. Receive tool output (Observation).
5. Inject observation back into context.
6. Repeat until the agent emits a final answer.

Component Breakdown

LangChain AgentExecutor The orchestration primitive. Uses `create_react_agent` with an `AgentExecutor` that handles tool dispatch, output parsing, and retry logic on malformed tool calls. The executor is configured with `max_iterations=50` and `handle_parsing_errors=True` to prevent hard stops on tool call formatting failures.

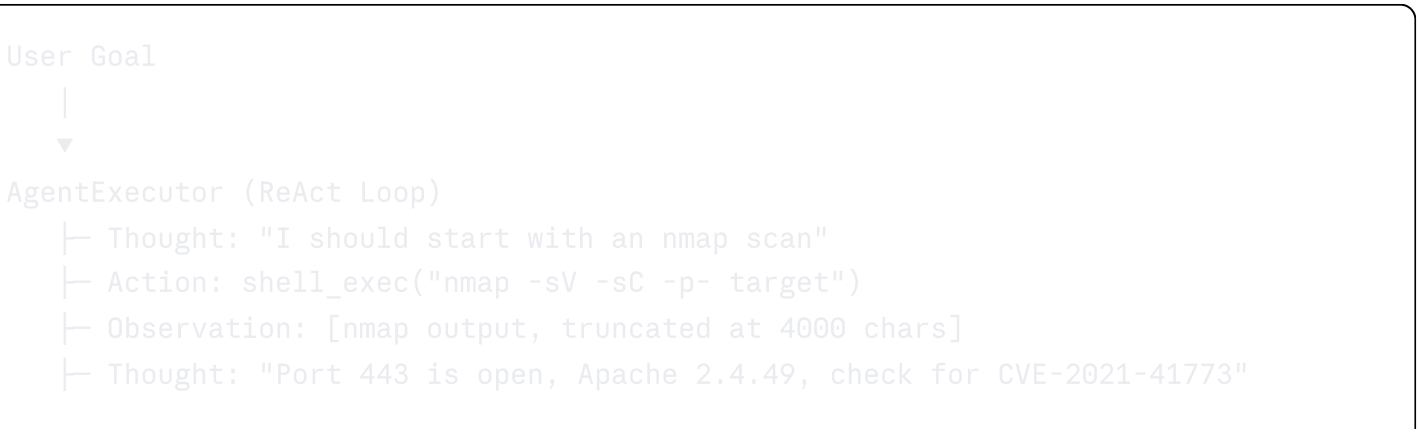
Shell Execution Tool A custom `BaseTool` subclass wrapping Python's `subprocess.run`. The tool accepts a shell command string, executes it with a configurable timeout (default 30s), and returns stdout + stderr as a combined string. There is no sandboxing at this stage — commands execute on the host directly. The tool includes naive output truncation (first 4000 chars) to prevent context overflow.

Web Search Tool A LangChain-wrapped SearxNG client. SearxNG is chosen because it is self-hostable (no API key dependency), supports multiple search backends simultaneously, and returns structured JSON. The LangChain `SearxSearchWrapper` is configured to return top-3 results per query.

Memory: ConversationSummaryMemory LangChain's `ConversationSummaryMemory` is used rather than buffer memory because tool outputs from shell commands can be extremely verbose. The memory component runs a secondary LLM call after each interaction turn to produce a compressed summary of prior context, which is prepended to the next prompt. This is intended to handle long engagements without hitting context limits.

Findings Accumulation There is no dedicated findings store. The agent is instructed via system prompt to maintain a running "findings log" in its output — an append-only text block that it updates as part of each reasoning step. This findings log is extracted from the final agent output via regex at session end.

Execution Workflow



```
    ├─ Action: searx_search("CVE-2021-41773 exploit PoC")
    ├─ Observation: [search results]
    ├─ Thought: "Found PoC, attempt path traversal"
    ├─ Action: shell_exec("curl http://target/cgi-bin/.%2e/.%2e/etc/passwd")
    ├─ Observation: [response]
    └─ ... (continues until max_iterations or goal satisfied)
```

Libraries & Tools

LangChain (langchain, langchain-community) Core orchestration framework. Used for `AgentExecutor`, `BaseTool`, `ConversationSummaryMemory`, prompt templates, and output parsers. LangChain is chosen at this stage because it provides the fastest path from "LLM + tools" to a running agent. Its `AgentExecutor` handles the ReAct loop plumbing without requiring custom orchestration code.

Limitations: LangChain's abstraction layers introduce non-obvious failure modes in tool error handling. The `handle_parsing_errors` flag silently retries malformed tool calls up to `max_iterations`, which can cause the agent to spin on a broken tool call rather than escalating. The `ConversationSummaryMemory` summary LLM call adds latency and introduces information loss — the summarizer may discard critical findings from earlier steps.

subprocess (Python stdlib) Used for shell command execution. `subprocess.run` with `shell=True`, `capture_output=True`, `timeout=30`. No process isolation, no PTY emulation, no interactive command support. Commands requiring interactive input will hang or fail silently.

Limitations: `shell=True` passes the command string to `/bin/sh`. More architecturally significant: there is no mechanism to handle long-running commands (nmap full-port scans, directory fuzzing) that exceed the timeout without losing output. The tool either returns truncated output or times out entirely, with no partial result recovery.

SearxNG (self-hosted) Meta-search engine used for CVE and exploit research. Configured with a local Docker deployment. SearxNG is preferred over external APIs because it eliminates external API dependencies and rate limits during long engagements.

Limitations: SearxNG result quality varies by backend availability. ExploitDB is not a default SearxNG backend — it requires custom engine configuration. Without targeted ExploitDB integration, exploit searches return SEO-heavy blog posts rather than actionable PoC commands. The tool has no mechanism to filter results by recency or source credibility.

Ollama / OpenAI API LLM backend. The LangChain `ChatOpenAI` and `ChatOllama` wrappers are used interchangeably, with local Mixtral-8x7B tested alongside GPT-4.

Limitations: Local models at this quantization level have significantly worse tool-call reliability than frontier models. Malformed JSON in tool calls is frequent, causing `handle_parsing_errors` to trigger retries that consume the iteration budget without making progress.

Failure Points

1. Context Collapse Under Long Tool Output Chains

The most severe structural failure. `ConversationSummaryMemory` summarizes prior turns, but the summarization is lossy and non-deterministic. In a 20+ step engagement, critical findings from early reconnaissance steps are frequently dropped from the summary because the summarizer LLM prioritizes recent context. The agent then re-enumerates services it already identified, wasting iteration budget on redundant work.

This is not a token limit problem — it is an **information architecture problem**. There is no typed, queryable store for findings. Everything lives in free-text conversation history, which degrades as a retrieval substrate over long engagements. You cannot query "what ports are open on 10.0.0.5" against a conversation buffer and get a reliable answer by step 40.

2. Single-Agent Deadlock on Blocked Commands

When a shell command blocks (interactive prompt, hung process), the entire agent halts. There is no watchdog, no timeout escalation path, no way to recover without restarting the session. The subprocess timeout kills the process, but the agent receives an empty or partial observation and typically retries the same command, hitting the timeout again. This is a **single-threaded execution trap**: because all reasoning and execution happen in one loop, any blocking operation freezes the entire system indefinitely until manual intervention.

3. No Approval Gate — Uncontrolled Command Execution

The agent executes any command its reasoning produces without human review. If the LLM hallucinates an aggressive or destructive command, it executes immediately. This is an **architectural control failure**: the system has no mechanism to distinguish exploratory from destructive commands before execution, and no way to inject user intent into the command selection process.

4. Findings Are Ephemeral and Non-Queryable

The "findings log" maintained in LLM output is unstructured text. It cannot be queried semantically, filtered by severity, or deduplicated across runs. If the same CVE is discovered in two separate tool calls, it appears twice with no consolidation. At session end, findings extraction via regex is brittle — any variation in LLM output format breaks the parser entirely.

5. No Replanning Capability

The ReAct loop has no mechanism to revise the high-level plan in response to findings. If the agent discovers that a planned approach is infeasible (e.g., the target is behind a WAF, or a service is on a non-standard port), it can only adapt at the individual tool-call level. There is no concept of task dependencies, task skipping, or goal decomposition into a DAG. The agent either powers through or exhausts its iteration budget against an infeasible approach.

6. Memory Reinitialization on Session Restart

If the agent process crashes or is restarted, all memory is lost. `ConversationSummaryMemory` is in-process only — there is no persistence layer. A 2-hour engagement that crashes at step 45 must restart from scratch. This makes the architecture unsuitable for any engagement lasting longer than a single uninterrupted process lifetime.

7. Tool Output Truncation Destroys Evidence

The naive character-limit truncation of tool output destroys information systematically. An nmap scan of 65535 ports produces output orders of magnitude larger than the truncation limit. The agent reasons only on the truncated prefix, missing open ports that appear later in the output. The architecture has no mechanism for chunked output processing, streaming reads, or structured output parsing — the tool is a black box that returns a string and loses everything beyond the truncation boundary.

Why This Is NOT the Final PDF Architecture

This approach shares with the PDF only the most surface-level resemblance: an LLM that executes shell commands. Every structural principle of the PDF is absent:

- **No agent separation**: The PDF uses five distinct agents with isolated responsibilities. Here, one agent does everything, making failure modes compound rather than isolate.
- **No shared memory coordination**: The PDF uses state.json, Mem0, and Qdrant as typed, queryable stores. Here, everything lives in conversation history — unstructured, lossy, and non-queryable.
- **No approval gate**: The PDF enforces a blocking user approval on every command. Here, commands execute without human review.
- **No task DAG**: The PDF decomposes goals into dependency-aware task graphs. Here, the agent improvises step-by-step with no structured plan.
- **No resilience chain**: The PDF has explicit failure escalation paths (Hacker → Searcher → Analyst → Supervisor). Here, failure means iteration budget exhaustion and a silent stop.
- **No parallelism**: The PDF runs multiple Hacker instances concurrently. Here, everything is strictly sequential.

The PDF architecture emerges precisely because every failure mode listed above was encountered in practice and required a structural solution, not a prompt engineering fix.

What the PDF Fixes Over This Approach

Failure in Approach 1	PDF Solution
Context collapse, lossy summarization	Mem0 entity graph + Qdrant vector store as external typed

Failure in Approach 1	PDF Solution
	memory
Single-agent deadlock on blocking commands	Separate HackerAgent coroutine; Analyst processes output asynchronously
No approval gate	`propose_command()` with blocking `asyncio.Event` per command
Ephemeral, non-queryable findings	Qdrant vector store with doc_type/severity filters; Mem0 typed entities
No replanning	Supervisor's `replan()` with full DAG rewrite capability
Memory lost on restart	state.json, Mem0, Qdrant are persistent external stores
Output truncation destroys evidence	Analyst receives full raw output; parses with structured LLM schema
No resilience chain	Explicit N+N retry budget with Searcher escalation and Analyst routing